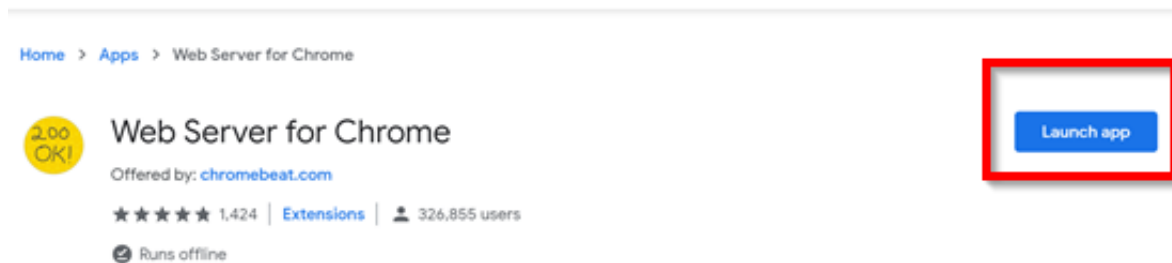


Learning Goals

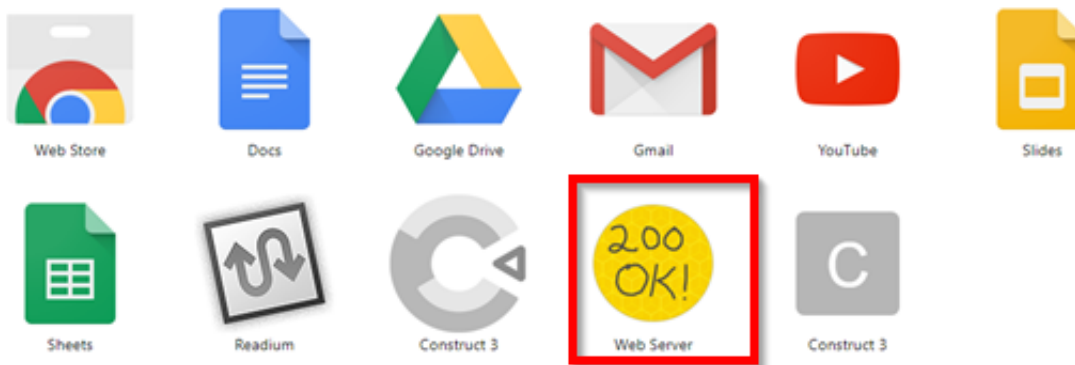
- Developer tools.
- Setting up a web server.
- Installing VS code.
- Downloading Phaser.

Obtaining the Chrome Web Server Browser Extension

Before we can jump in and start coding there are a few setup tasks that we need to complete. The first thing we need to do is have a local web server running on our machine. We will be using a Chrome browser extension to setup our web server. You are welcome to use another route as far as the webserver goes, but this course will only be covering the Chrome browser extension we use in this course. In order to get the Chrome extension you will need to visit this link here to obtain it: <https://chrome.google.com/webstore/detail/web-server-for-chrome/ofhbbkphhbklhfoeikjpcbhmlcggigb?hl=en>



This is a very simple browser extension that you just simply point it to a local folder where you want to host files and then it will spin up your server for you. Once you have the extension added to your Chrome browser you will see it in the Chrome apps menu:



In order to use the extension you will just click on the icon, and a new window will open:



Web Server for Chrome

Please [leave a review](#) to help others find this software.

CHOOSE FOLDER Current: /phaser3



Web Server: STARTED

Web Server URL(s)

- <http://127.0.0.1:8887>

Options (needs restart)

- ☐ Run in background
 - ☐ Start on login
- ☐ Accessible on local network
 - ☐ Also on internet
- ☐ Prevent computer from sleeping
- ☒ Automatically show index.html

Enter Port

8887

[Show Advanced Settings](#)

Need to [Report a problem](#)? Open source, MIT license.

All you need to do now is click on **“Choose Folder”**:

CHOOSE FOLDER

Current: /phaser3

Web Server: STARTED

Web Server URL(s)

- <http://127.0.0.1:8887>

So once you select the folder, all the assets that are in that folder will be made available on the server. Then if you click the link provided:

Web Server URL(s)

- <http://127.0.0.1:8887>

You will see a list of all the current assets in that directory:



Index of current directory...

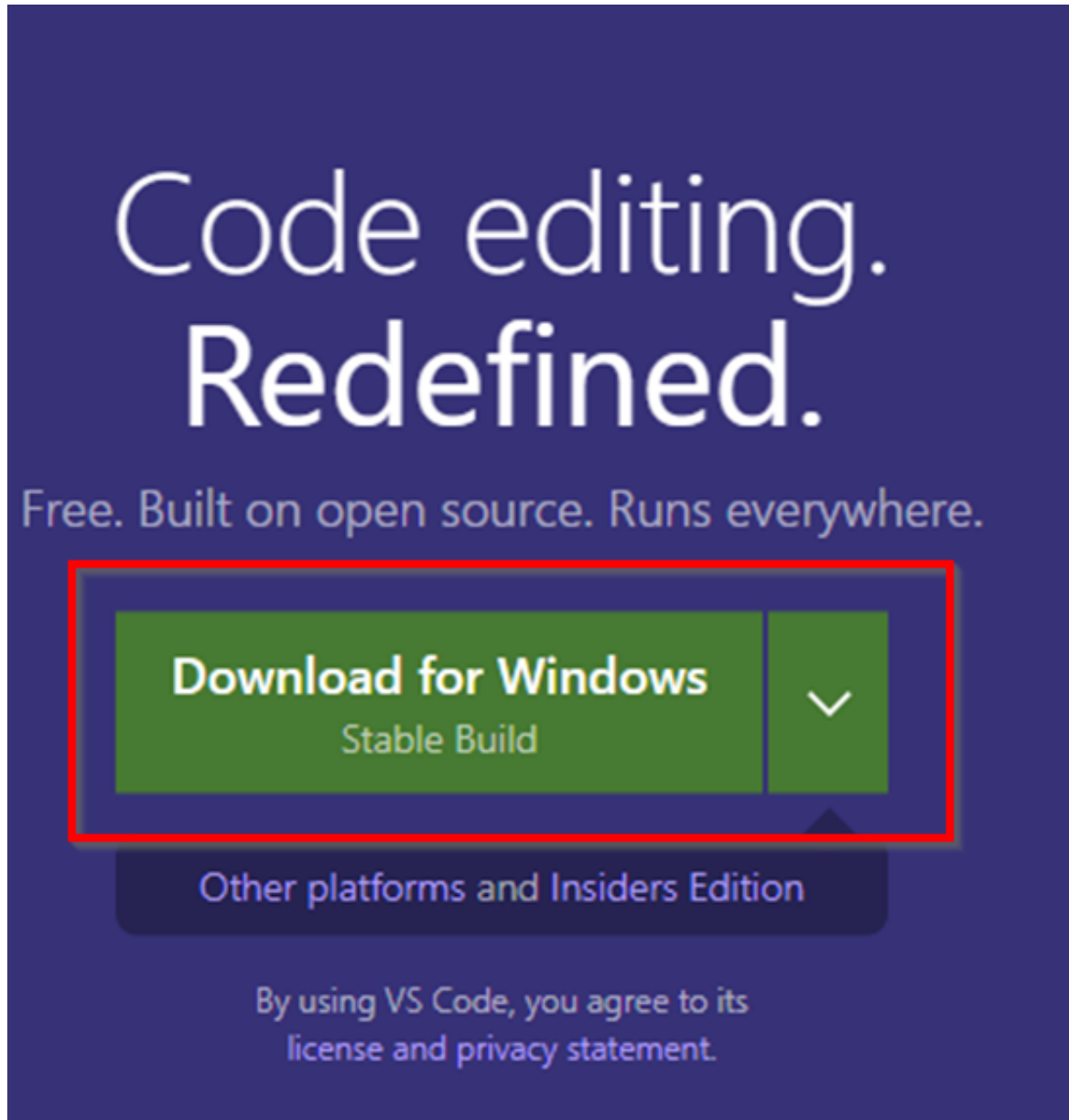
Name	Size	Date Modified
 [parent directory]		



Currently the directory is empty so that is why it is showing the way it is. The second tool that you are going to need is an Editor. There are many free editors out there to download and use.

Obtaining the Code Editor

We will be using VS Code in this course. Again you are welcome to use any editor of your choice, but we will only be covering VS Code in the course. In order to download VS Code you can visit this link here: <https://code.visualstudio.com/>



Once you have the installer downloaded all you need to do is run the installer and follow all the on-screen prompts for the installation.

Obtaining the Phaser Library

The final tool we need now is the actual Phaser 3 library. You can visit this link to obtain the Phaser library: <https://phaser.io/download/stable>



At the time of this course the latest version of Phaser is Phaser 3.16.2 “Ishikawa.” There are a few different ways to obtain the Phaser library from here:



You can download the **min.js** file.

Summary

Developing With Phaser

- In order to start developing with Phaser you will need to setup a web server to host your files.
- In addition to the web server, you will need to have an editor for writing your code. There are many options available each with their own pros and cons.

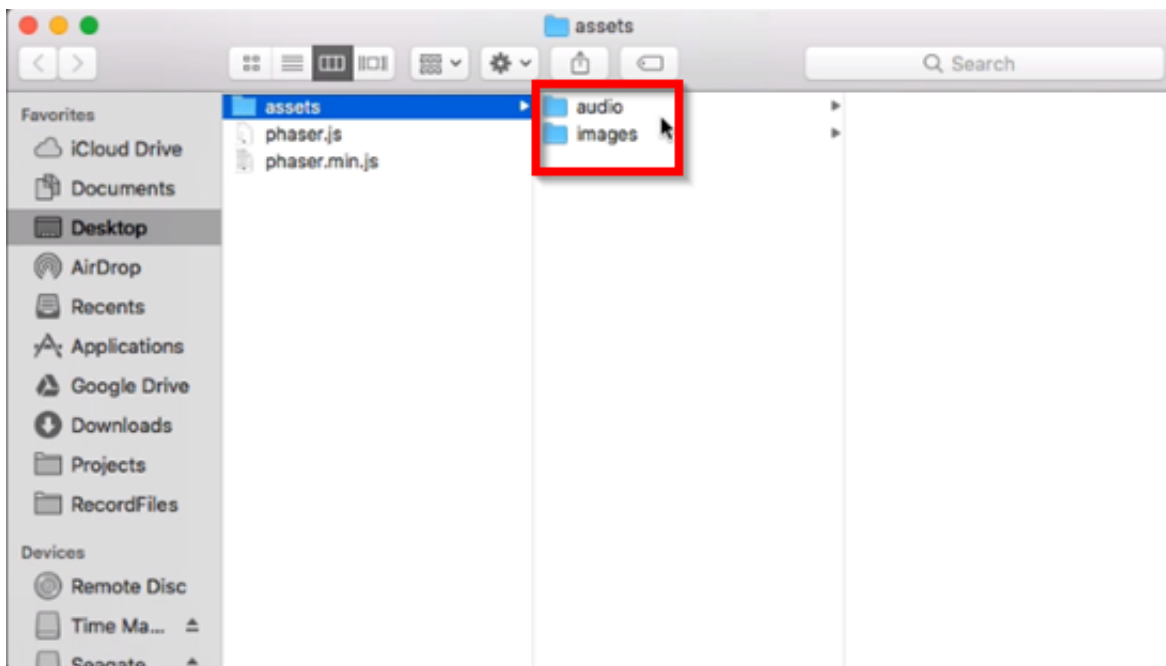


The instructions in the video for this particular lesson have been updated, please see the lesson notes below for the corrected code.

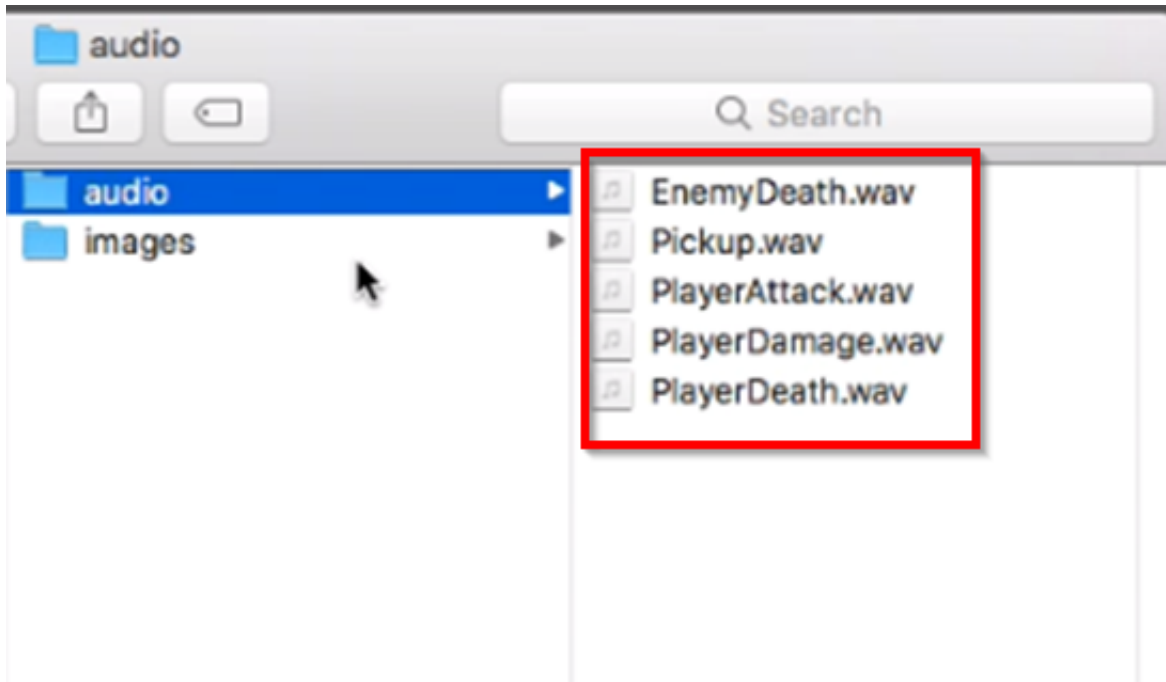
Learning Goals

- Basic project setup.
- Creating a basic HTML page.
- Running the server.

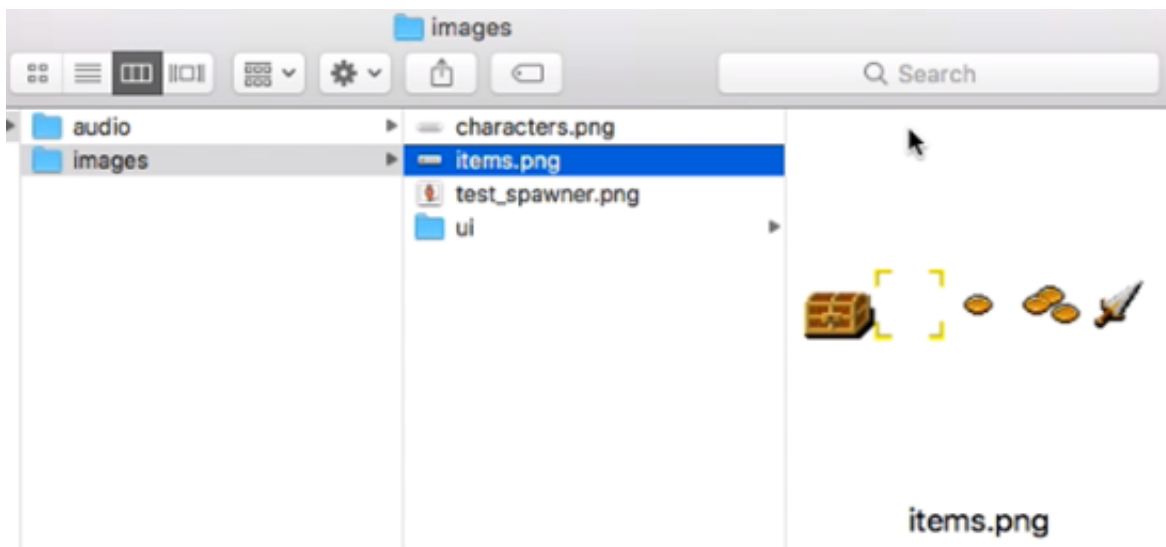
In this lesson we will begin by setting up our project structure. You should begin this lesson by first downloading the course assets, and you can download these assets from the **course home** page. Once you have downloaded and extracted the assets folder you should see two folders called **“assets”** and **“images”**:



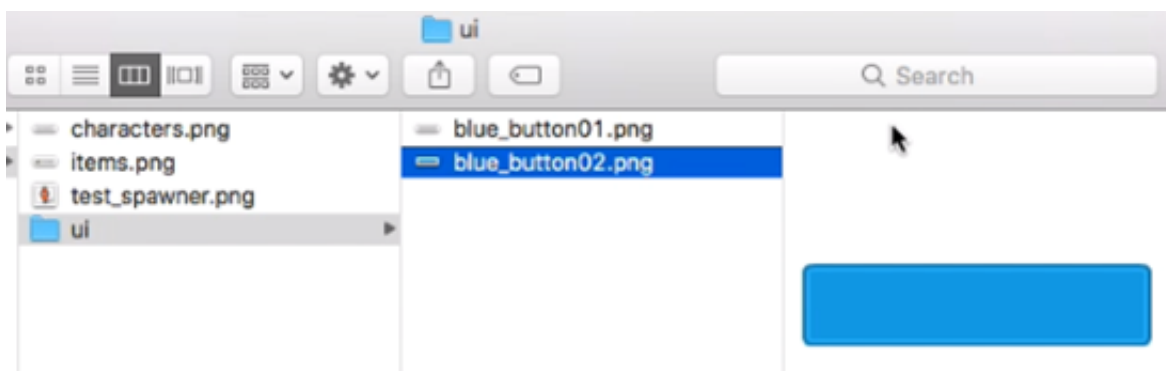
The audio folder has five different sound effects in it, we will use these in our game for the player attack, when the player takes damage, etc:



In the images folder there are a few sprite sheets, one of them is characters, the other one is items:

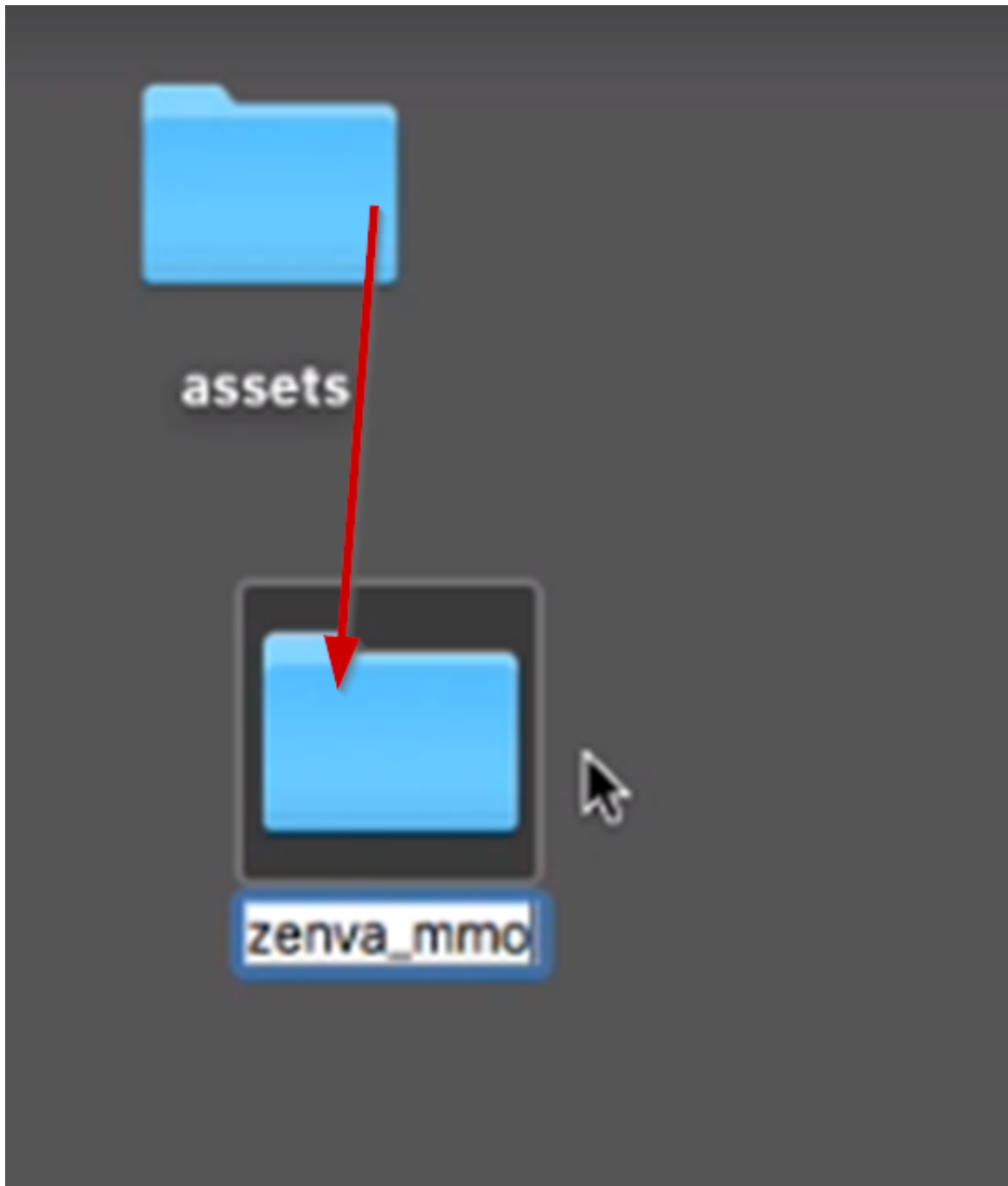


There is also a test spawner image, and a UI folder that contains two buttons for us to use in our game:

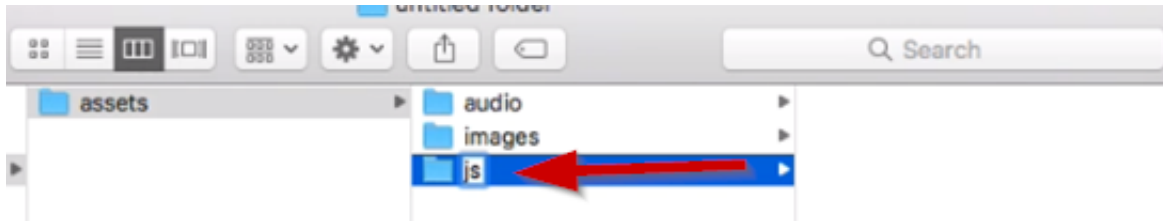




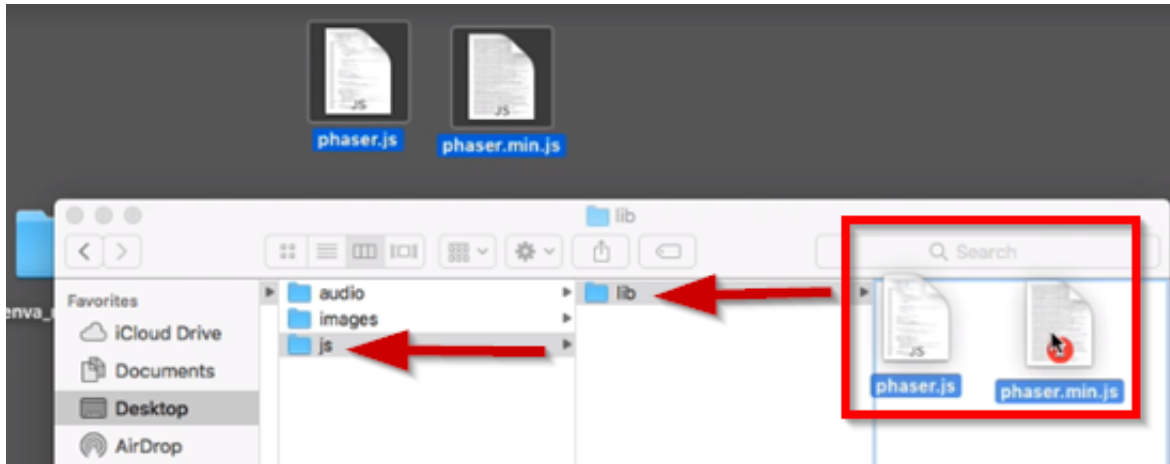
In order to start setting up our project structure we will need to create a new folder and call it **“zenva_mmorpg”** once you have this folder created you can place the assets folder inside this folder:



We now need to create a new folder inside the assets folder that is now in the zenva_mmorpg folder and call the folder **“js”**:

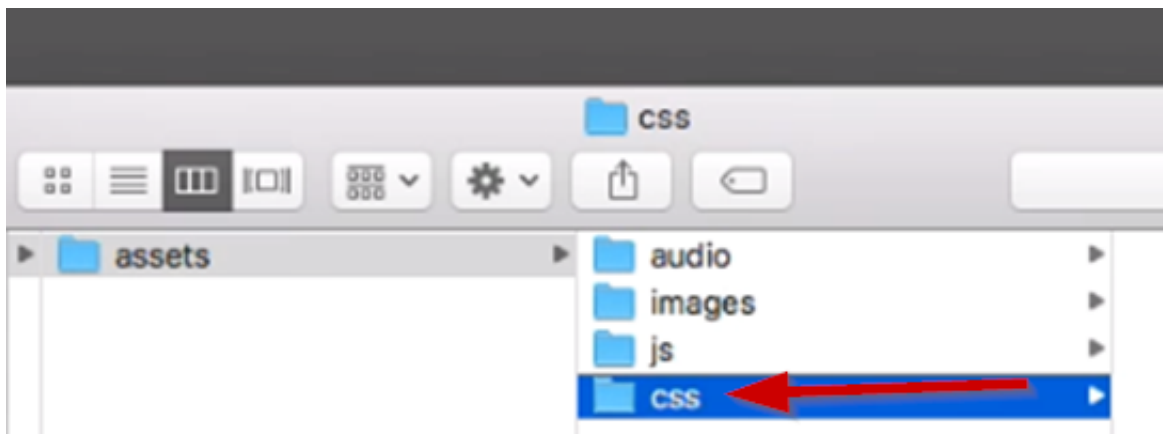


Create a new subfolder in the js folder called **“lib”**:

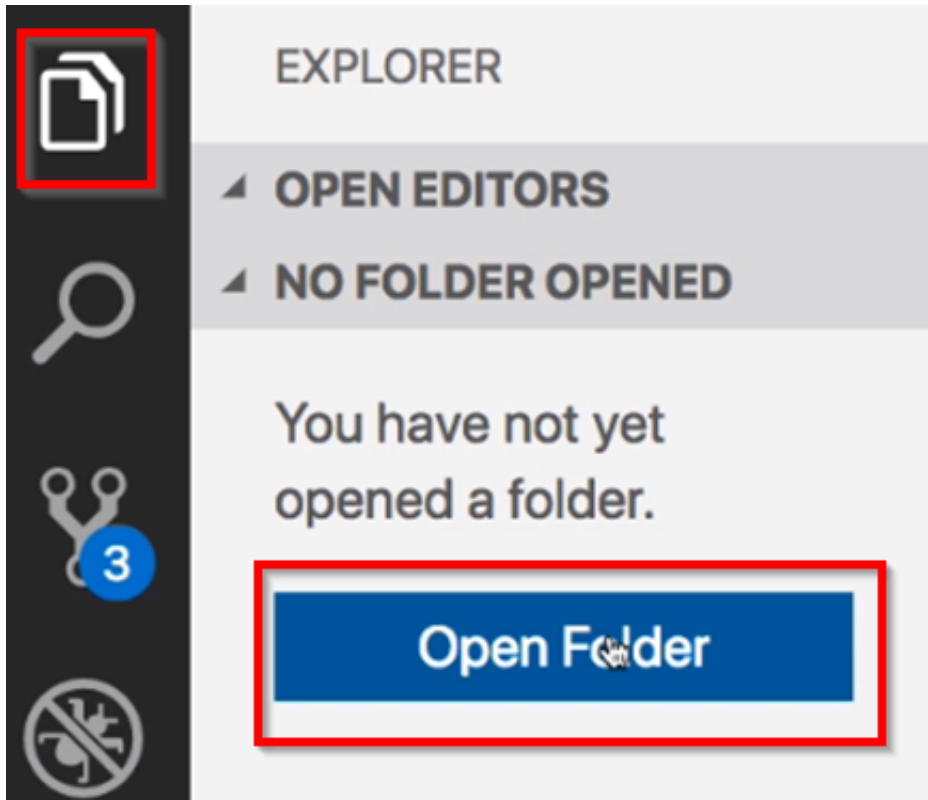


The following instructions have been updated, and differs from the video:

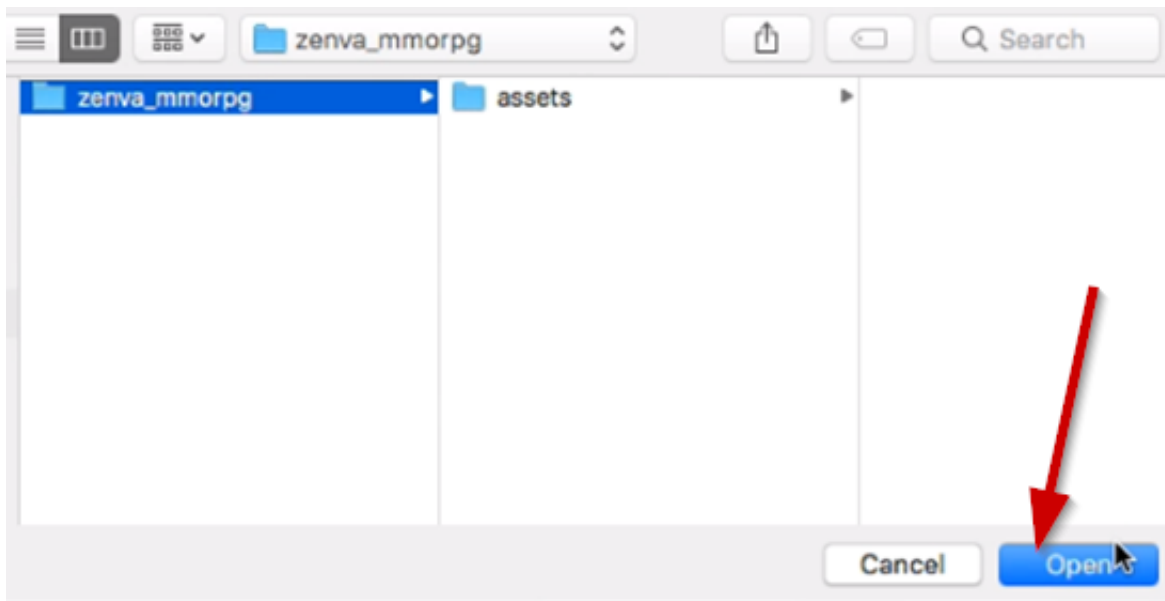
The lib folder will be used to keep track of any external libraries that we wanna use for our game. We will need Phaser inside of the lib folder. For the time being we are going to include both the Phaser and the Phaser minified file. You can download both files from the [Phaser download page](#). One other folder that we need to make inside the assets folder is the **“css”** folder:



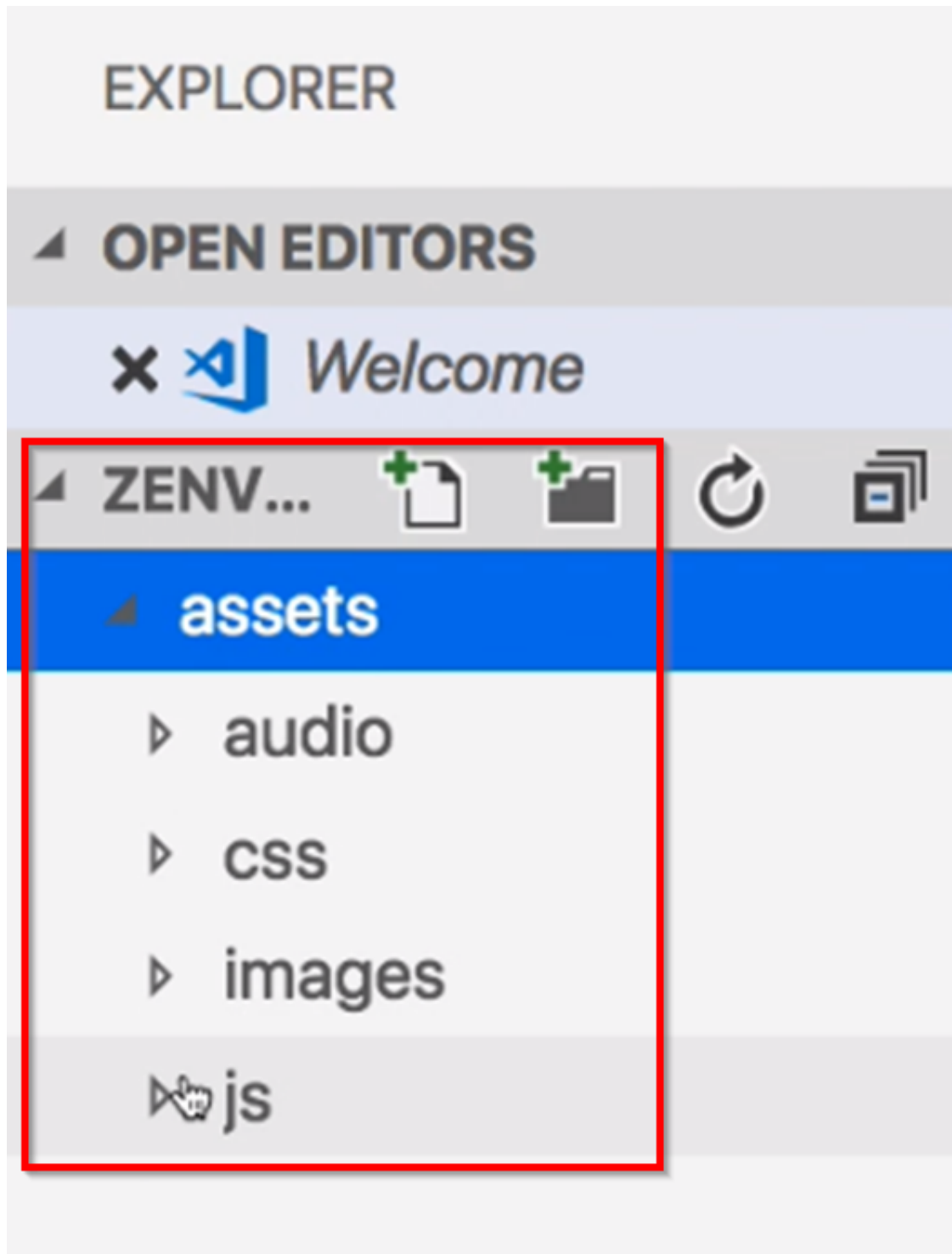
We will use this folder to keep track of any external CSS files. We now have the basic structure of our project in place and now we can open up the folder inside the code editor. One way to do this in VS Code is when you open VS code without any folders selected you can click on the File Icon in the upper left hand corner and then choose Open Folder:



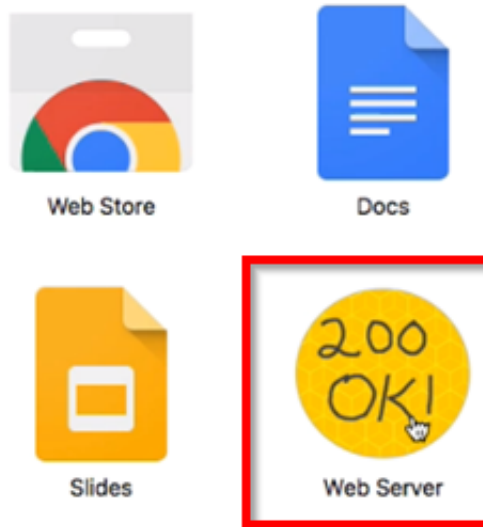
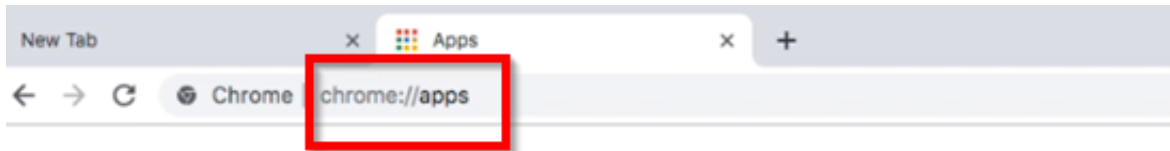
Once you choose Open Folder you will need to navigate to where you have the folder you just created in the previous steps:



This will reload the code editor once you open the zenva_mmorpg folder in VS Code. You will see that we have our assets in here now, and they are listed on the left side of the editor:



Next we need to start up the Webserver for hosting our files, open the Chrome browser and click on Apps:



Once the Chrome webserver has been opened click on Choose Folder:



Web Server for Chrome

Please [leave a review](#) to help others find this software.

CHOOSE FOLDER

Current: /zenva_mmorpg

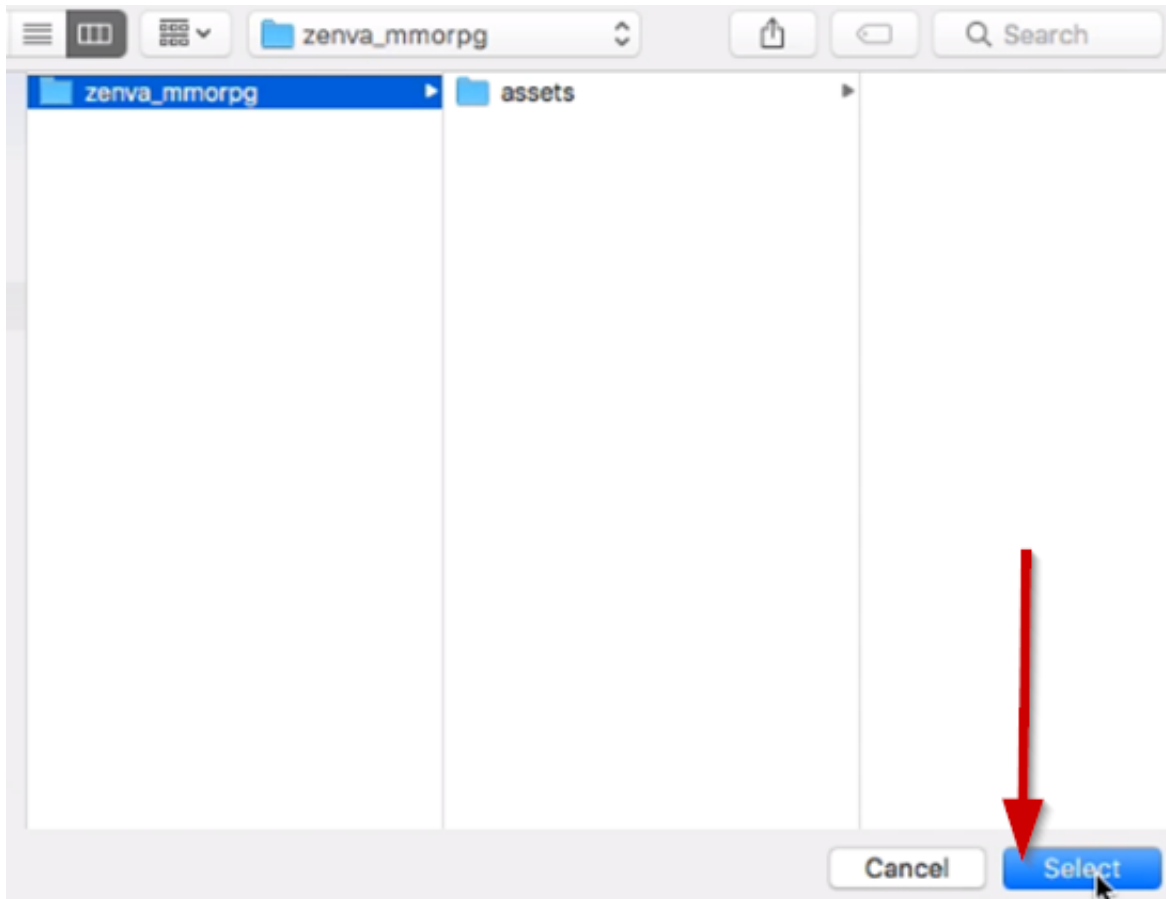


Web Server: STARTED

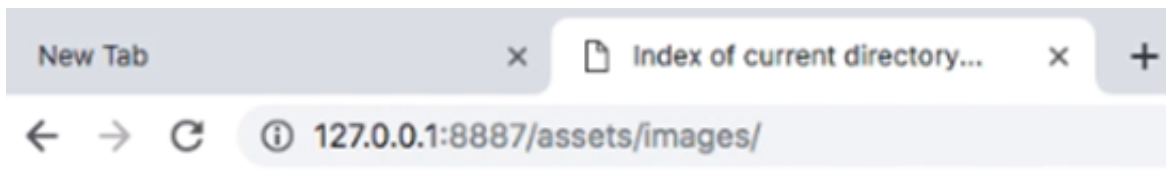
Web Server URL(s)

- <http://127.0.0.1:8887>

You will then choose the zenva_mmorpg folder you just created previously:



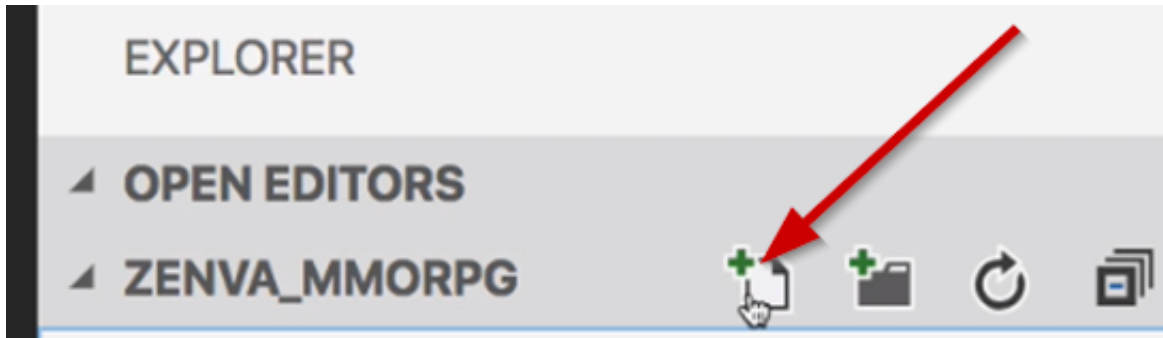
This will be the root of our project. So now when we click on the link, you will see that the **localhost 8887** is now hosting our static files. You will see that we have the assets folder and images:



Index of current directory...

Name	Size	Date Modified
[parent directory]		
characters.png		
items.png		
test_spawner.png		
ui/		

You can switch back to the code editor now, and next we need to create an HTML page. Because this is a JavaScript library, we will be including the JavaScript files in our HTML page. The HTML page will be what is rendered to the end user when they visit our site. Create a new file:



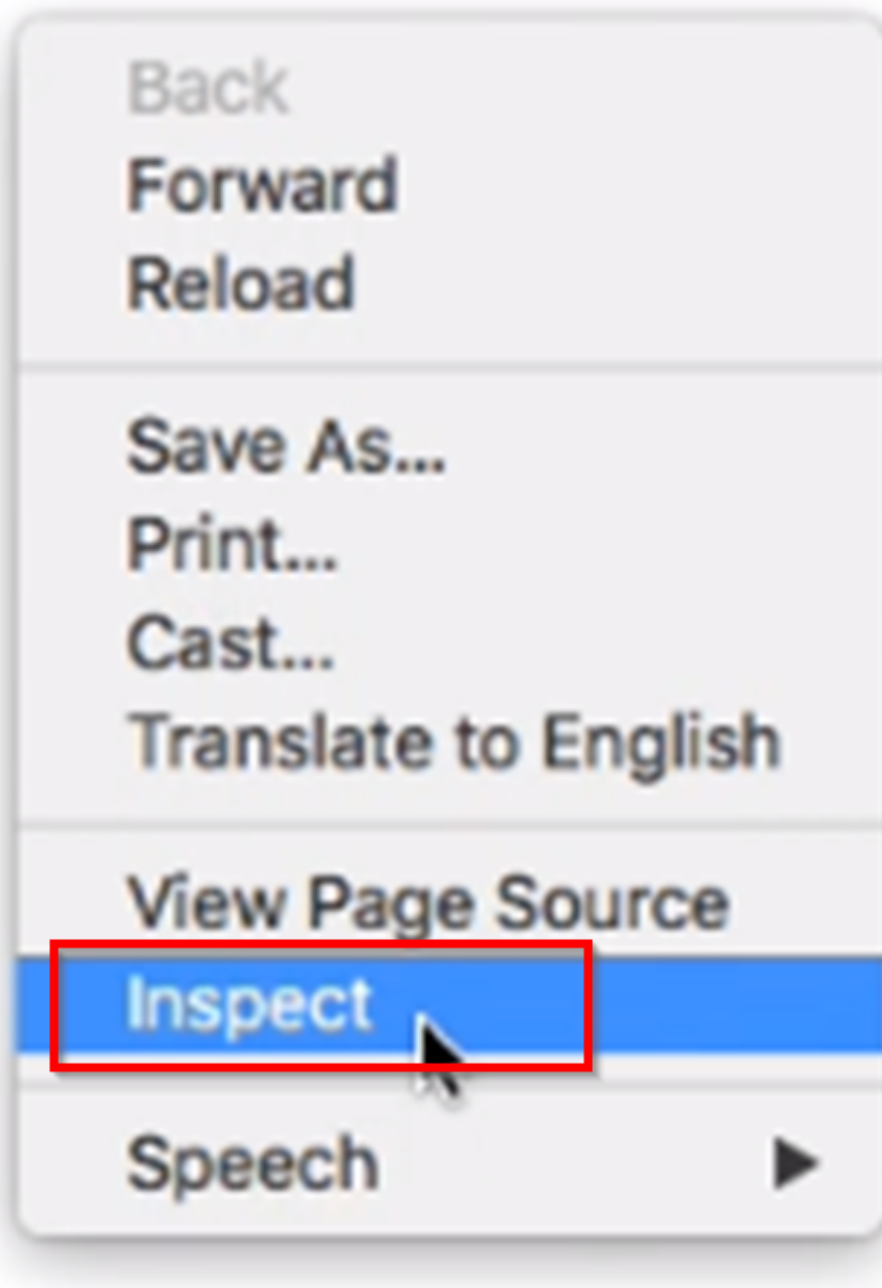
Name the file **"index.html"** We then need to create the basic structure of the HTML page, so open this file in the code editor. See the code below and follow along:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="user-scalable=0, initial-scale=1, minimum-
scale=1, maximum-scale=1, width=device-width, minimal-ui=1">
    <title>Zenva MMORPG</title>
  </head>
  <body>
    <div id="phaser-game"></div>
    <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
  </body>
</html>
```

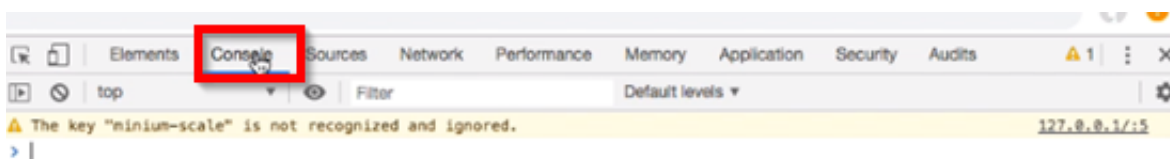
Save this file. Switch back to the browser and refresh the page, you will see a few things and the first is that our asset list has went away. The reason for this is that when a server detects an **index.html** page, the server will render that page at the root path. You can see that our index page is being rendered, but we don't have any content currently, but if you look at the title on the tab you can see that it has the tile name we specified:



Right click on the webpage and choose inspect:



Click on the **Console**:



Summary



Phaser Library

- In order to create your game, you will need to include the Phaser library. The library is available via direct download, on a CDN, NPM, and on GitHub.
- When you are developing your game, it is important to keep an eye on the version of Phaser you are developing against. Some features may not be backwards compatible.



Now to start adding content to our empty page!

In Visual Studio Code (VSC) go into your *js* folder and create a new file called **main.js**. In your *index.html* we need to reference this new file. As the last line in the `<body>` section, add this line:
`<script src="assets/js/main.js" charset="utf-8"></script>`

Back in **main.js** we will start to fill in some code. The first thing is to create a Phaser game config object. This lets Phaser know how you would like to run your game:

```
var config = {  
    type: Phaser.AUTO,  
    width: 800,  
    height: 600,  
    scene: {  
        preload: preload,  
        create: create,  
    },  
};
```

Now we have enough to create a Phaser game object, passing in the config object. While we're at it, let's create stub functions for **preload()** and **create()**:

```
var game = new Phaser.Game(config);  
  
function preload() {  
  
}  
  
function create() {  
  
}
```

If you save your files and then refresh the tab that's running the webserver, you should see a black canvas that is 800 x 600 pixels. Progress!

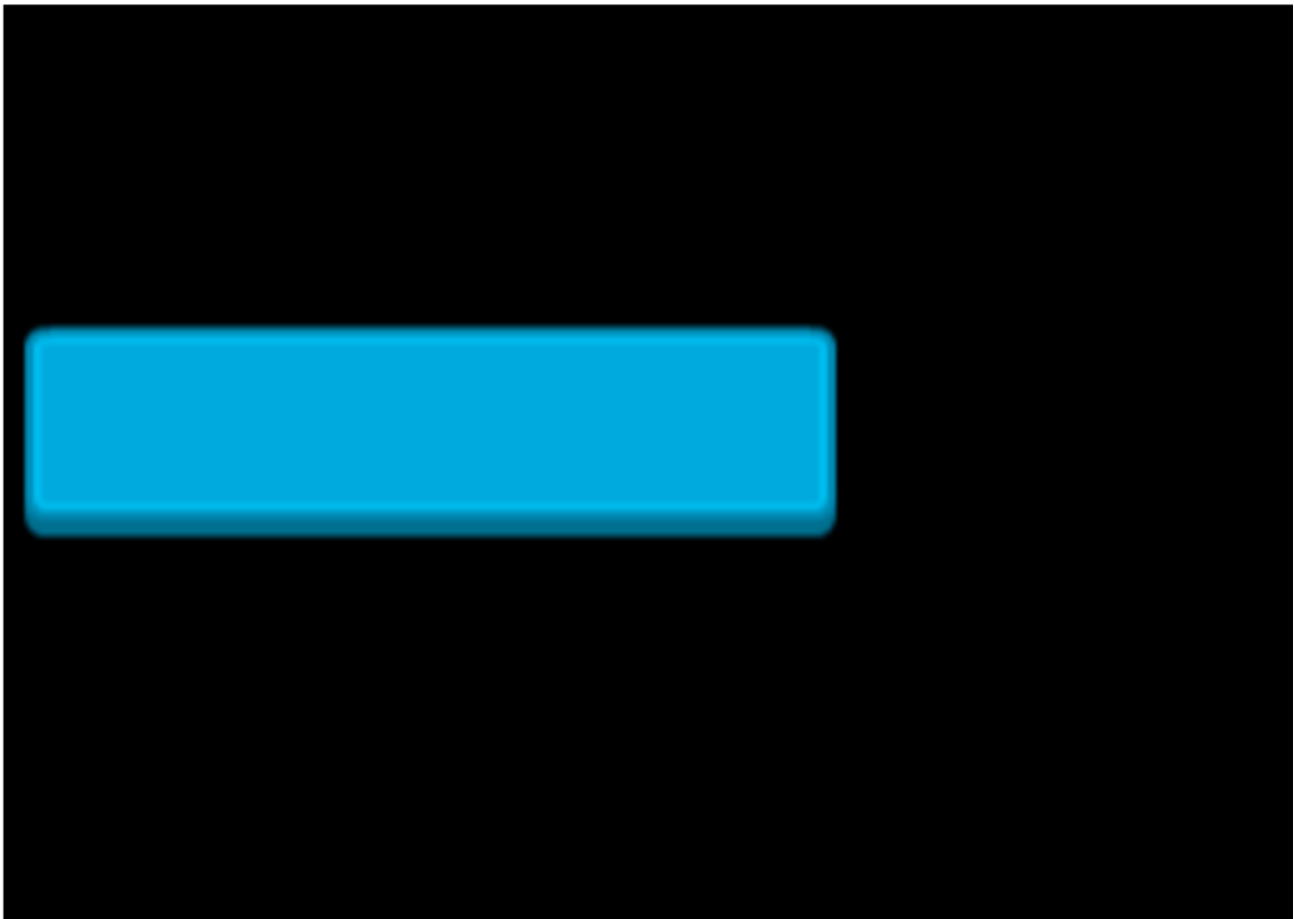


Now to put something on our black canvas.

We'll add code to **preload()** to load in an image and then in **create()** we'll add the image to the game:

```
function preload() {  
    this.load.image('button1', 'assets/images/ui/blue_button01.png');  
}  
  
function create() {  
    this.add.image(100, 100, 'button1');  
}
```

If you run this now, you should see this in the upper left corner:



Why is the button all the way left when we are specifying an x,y position of 100,100?

The answer is that, by default, Phaser has the origin of a graphic be in its center. To change this, we need to change the origin:

```
function create() {  
    var button = this.add.image(100, 100, 'button1');  
    button.setOrigin(0,0);  
}
```



Now if you refresh you should see:



We also could have loaded our graphic as a sprite instead of as an image. The primary difference is that images can't be animated and sprites can.



Now we will learn to deal with a graphic as a spritesheet. To do so, you load it in as a spritesheet and pass in an object that has information about the sheet. Here we are just passing in the frame width and height:

```
function preload() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    // the next line is NEW
    this.load.spritesheet('items', 'assets/images/items.png', {frameWidth:32, frameHeight:32});
}
```

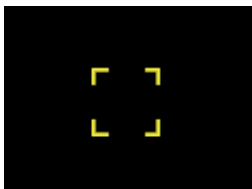
To display a single frame of the sheet, you still use the **add.image()** function, but you pass in the index of the frame to display. (They are numbered starting at zero).

This will display frame 1 of the sheet:

```
function create() {
    var button = this.add.image(100, 100, 'button1');
    button.setOrigin(0,0);

    this.add.image(300, 300, 'items', 1); // NEW
}
```

When you refresh the game, you should see this square cursor:



If you don't specify a frame index, Phaser will simply display the first frame of the sheet.



Phaser can use a number of different physics plugins. Here, we will use Arcade Physics, a simple plugin, well-suited for mobile. Modify the config object to use the plugin and specify some plugin config information. Be sure there is a comma after the scene block:

```
var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scene: {
    preload: preload,
    create: create,
  },
};
```

Then add:

```
physics: {
  default: 'arcade',
  arcade: {
    debug: true,
    gravity: {
      y: 5,
    },
  },
},
```

Refresh and you should see a new button slowly dropping to the bottom of the screen.

After that, change the gravity to 0 and we are done for now.

You can also add a physics sprite to the game by using `this.physics.add.sprite()` .

The full config should look like this:

```
var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scene: {
    preload: preload,
    create: create,
  },
  physics: {
    default: 'arcade',
    arcade: {
      debug: true,
      gravity: {
        y: 5,
      },
    },
  },
};
```



```
};  
},
```



In order to focus on player input we need something to move!

Load a new spritesheet that has the characters for our game:

```
function preload() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    this.load.spritesheet('items', 'assets/images/items.png', {frameWidth:32, frameHeight:32});
    // next line is NEW
    this.load.spritesheet('characters', 'assets/images/characters.png', {frameWidth:32, frameHeight:32});
}
```

Then in **create()** add the first frame to the canvas and scale it up 2x normal size:

```
function create() {
    var button = this.add.image(10, 100, 'button1');
    button.setOrigin(0,0);

    this.add.image(300, 300, 'items', 1);

    this.physics.add.image(500, 100, 'button1');

    this.player = this.physics.add.image(32, 32, 'characters', 0); // NEW
    this.player.setScale(2); // NEW
}
```

Now to start responding to cursor control. Phaser has a convenient built-in object that listens for the arrow keys. We'll use that. At the bottom of your **create()** function add this line: `this.cursors = this.input.keyboard.createCursorKeys();`

That's the first step, but we still need a way to respond. We'll create an update function. Modify your config object like this (the update line is new):

```
scene: {
    preload: preload,
    create: create,
    update: update,
},
```

Now go to the bottom of the file and define the **update()** function. We'll put in some debug messages so you can test it:

```
function update() {

    if(this.cursors.left.isDown) {
        console.log("left");
    } else if (this.cursors.right.isDown) {
        console.log("right");
    }
}
```



```
}

if(this.cursors.up.isDown) {
    console.log("up");
} else if (this.cursors.down.isDown) {
    console.log("down");
}
}
```

If you save and refresh your game (and open the console) you should see debug messages when you press the arrow keys.

To actually see something moving, we need to change the velocity of the player sprite so replace the debug messages and also add a line to initially set velocity to zero each frame:

```
function update() {
    this.player.setVelocity(0);

    if(this.cursors.left.isDown) {
        this.player.setVelocityX(-160);
    } else if (this.cursors.right.isDown) {
        this.player.setVelocityX(160);
    }

    if(this.cursors.up.isDown) {
        this.player.setVelocityY(-160);
    } else if (this.cursors.down.isDown) {
        this.player.setVelocityY(160);
    }
}
```

When you refresh and run now you should be able to move the sprite around with the arrows!



There are two collision issues we need to address. One is that the player can move outside the bounds of the game. The second is that the player doesn't collide with other objects.

For the first issue, Phaser has a convenience method handy. Add it to your **create()** function:

```
this.player = this.physics.add.image(32, 32, 'characters', 0);  
this.player.setScale(2);  
this.player.body.setCollideWorldBounds(true); // NEW
```

Next, we are going to have the player collide with the physics button you created earlier. In your config object make sure Y gravity is set to zero.

Then in **create()** we will make the button immobile and create a collider with the player like so:

```
function create() {  
    var button = this.add.image(10, 100, 'button1');  
    button.setOrigin(0,0);  
  
    this.add.image(300, 300, 'items', 1);  
  
    this.wall = this.physics.add.image(500, 100, 'button1'); // NEW  
    this.wall.setImmovable(); // NEW  
  
    this.player = this.physics.add.image(32, 32, 'characters', 0);  
    this.player.setScale(2);  
    this.player.body.setCollideWorldBounds(true);  
  
    this.physics.add.collider(this.player, this.wall); // NEW  
  
    this.cursors = this.input.keyboard.createCursorKeys();  
}
```

Test your game and you will not be able to move onto the button.

Phaser also has this concept of an **overlap**, which is detecting when two objects are overlapping, but not affecting them with physics.

Let's detect when the player overlaps with a chest. Create a chest variable and set it to the items image. Note that the item frame should be zero. Then, below the player/wall collider create a player/chest overlap detector with an anonymous function printing a message:

```
function create() {  
    var button = this.add.image(10, 100, 'button1');  
    button.setOrigin(0,0);  
  
    this.chest = this.physics.add.image(300, 300, 'items', 0); // NEW  
  
    this.wall = this.physics.add.image(500, 100, 'button1');  
    this.wall.setImmovable();  
  
    this.player = this.physics.add.image(32, 32, 'characters', 0);
```



```
this.player.setScale(2);
this.player.body.setCollideWorldBounds(true);

this.physics.add.collider(this.player, this.wall);
this.physics.add.overlap(this.player, this.chest, function() { console.log('overlap');}); // NEW

this.cursors = this.input.keyboard.createCursorKeys();
}
```



Before you can trigger a sound you have to load it, so in **preload()** load in a sound we will use for pickup. Note that we are specifying an array, where we could pass in several different audio formats and allow the system to choose the best one for the browser:

```
function preload() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    this.load.spritesheet('items', 'assets/images/items.png', {frameWidth:32, frameHeight:32});
    this.load.spritesheet('characters', 'assets/images/characters.png', {frameWidth:32, frameHeight:32});
    this.load.audio('goldSound', ['assets/audio/Pickup.wav']); // NEW
}
```

Next we will set it up so that the sound will play one time when the player overlaps the chest and then the chest is destroyed. We can do this with two lines:

```
function create() {
    var goldPickupAudio = this.sound.add('goldSound'); // NEW

    var button = this.add.image(10, 100, 'button1');
    button.setOrigin(0,0);

    this.chest = this.physics.add.image(300, 300, 'items', 0);

    this.wall = this.physics.add.image(500, 100, 'button1');
    this.wall.setImmovable();

    this.player = this.physics.add.image(32, 32, 'characters', 0);
    this.player.setScale(2);
    this.player.body.setCollideWorldBounds(true);

    this.physics.add.collider(this.player, this.wall);
    this.physics.add.overlap(this.player, this.chest, function(player, chest) { goldPickupAudio.play(); chest.destroy(); }); // NEW

    this.cursors = this.input.keyboard.createCursorKeys();
}
```



Now we will create some different scenes, such as a **boot** scene to load assets, and we will create some classes to represent, for example, the player and the chest.

In Visual Studio Code create two subfolders in your **js** folder: **scenes** and **classes**. In the **scenes** subfolder create four new files: **BootScene.js**, **TitleScene.js**, **UiScene.js**, and **GameScene.js**.

Likewise, in the **classes** subfolder create **Player.js** and **Chest.js**.

The naming convention is that when you create a js class file you give it the same name as the class.

Now open up **BootScene.js** and create the **BootScene** class, which extends **Phaser.Scene**. Add a generic constructor and make two stub functions that will automatically be called by Phaser:

```
class BootScene extends Phaser.Scene {  
  
    constructor() {  
        super('Boot');  
    }  
  
    preload() {  
  
    }  
  
    create() {  
  
    }  
}
```

Copy this entire shell into **GameScene.js**, **TitleScene.js** and **UiScene.js**. You will need to customize each file to reflect the different class names. Also you will need to change the name you pass in to the Phaser constructor. Here is how **GameScene.js** is changed:

```
class GameScene extends Phaser.Scene {  
  
    constructor() {  
        super('Game');  
    }  
  
    preload() {  
  
    }  
  
    create() {  
  
    }  
}
```

Go on to change **TitleScene.js** to reflect **'TitleScene'** and **'Title'**, and change **UiScene.js** to reflect **'UiScene'** and **'Ui'**.

Now go back into **main.js** and copy **preload()**, **create()** and **update()**. Paste those into the



BootScene class. You will need to remove the **function** keyword, like so:

```
class BootScene extends Phaser.Scene {

    constructor() {
        super('Boot');
    }

    preload() {
        this.load.image('button1', 'assets/images/ui/blue_button01.png');
        this.load.spritesheet('items', 'assets/images/items.png', {frameWidth:32, frameHeight:32});
        this.load.spritesheet('characters', 'assets/images/characters.png', {frameWidth:32, frameHeight:32});
        this.load.audio('goldSound', ['assets/audio/pickup.wav']);
    }

    create() {
        var goldPickupAudio = this.sound.add('goldSound');

        var button = this.add.image(10, 100, 'button1');
        button.setOrigin(0,0);

        this.chest = this.physics.add.image(300, 300, 'items', 0);

        this.wall = this.physics.add.image(500, 100, 'button1');
        this.wall.setImmovable();

        this.player = this.physics.add.image(32, 32, 'characters', 0);
        this.player.setScale(2);
        this.player.body.setCollideWorldBounds(true);

        this.physics.add.collider(this.player, this.wall);
        this.physics.add.overlap(this.player, this.chest, function(player, chest) { goldPickupAudio.play(); chest.destroy(); });

        this.cursors = this.input.keyboard.createCursorKeys();
    }

    update() {
        this.player.setVelocity(0);

        if(this.cursors.left.isDown) {
            this.player.setVelocityX(-160);
        } else if (this.cursors.right.isDown) {
            this.player.setVelocityX(160);
        }

        if(this.cursors.up.isDown) {
            this.player.setVelocityY(-160);
        } else if (this.cursors.down.isDown) {
            this.player.setVelocityY(160);
        }
    }
}
```



```
}
```

Go ahead and delete **preload()**, **create()** and **update()** from **main.js**. Now we will change the config object to recognize our new scenes. The new section starts with **scene** and ends with **],**:

```
var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scene: [
    BootScene,
    TitleScene,
    GameScene,
    UiScene,
  ],
  physics: {
    default: 'arcade',
    arcade: {
      debug: true,
      gravity: { y:0, },
    },
  },
};
```

We still need to let the HTML know where to find these js scenes. In your index.html file modify the `<body>` tag to include the four new js files. The new lines are between the first and last `<script>` tags:

```
<body>
  <div id="phaser-game"> </div>
  <script src="//cdn.jsdelivr.net/npm/phaser@3.21.0/dist/phaser.js"></script>
  <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
  <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
  <script src="assets/js/main.js" charset="utf-8"></script>
</body>
```

If you save and refresh your scene should run exactly as before, only now a lot has changed under the hood. The reason it's working as before is that by default Phaser loads in the first scene in the config object.



Go ahead and cut and paste the **create()** and **update()** methods from BootScene into GameScene. GameScene should now look like this:

```
class GameScene extends Phaser.Scene {

    constructor() {
        super('Game');
    }

    create() {
        var goldPickupAudio = this.sound.add('goldSound');

        var button = this.add.image(10, 100, 'button1');
        button.setOrigin(0,0);

        this.chest = this.physics.add.image(300, 300, 'items', 0);

        this.wall = this.physics.add.image(500, 100, 'button1');
        this.wall.setImmovable();

        this.player = this.physics.add.image(32, 32, 'characters', 0);
        this.player.setScale(2);
        this.player.body.setCollideWorldBounds(true);

        this.physics.add.collider(this.player, this.wall);
        this.physics.add.overlap(this.player, this.chest, function(player, chest) { goldPickupAudio.play(); chest.destroy(); });

        this.cursors = this.input.keyboard.createCursorKeys();
    }

    update() {
        this.player.setVelocity(0);

        if(this.cursors.left.isDown) {
            this.player.setVelocityX(-160);
        } else if (this.cursors.right.isDown) {
            this.player.setVelocityX(160);
        }

        if(this.cursors.up.isDown) {
            this.player.setVelocityY(-160);
        } else if (this.cursors.down.isDown) {
            this.player.setVelocityY(160);
        }
    }
}
```

Modify BootScene so that it has a new **create()** method that loads the GameScene:

```
class BootScene extends Phaser.Scene {
```



```
constructor() {
    super('Boot');
}

preload() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    this.load.spritesheet('items', 'assets/images/items.png', {frameWidth:32, frameHeight:32});
    this.load.spritesheet('characters', 'assets/images/characters.png', {frameWidth:32, frameHeight:32});
    this.load.audio('goldSound', ['assets/audio/pickup.wav']);
}

create() {
    this.scene.start('Game');
}
```

If you save and refresh your game should *still* work exactly the same.



Now we will work on separating the logic of the player and the chest.

In **Player.js** add the following class definition and constructor:

```
class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);
  }
}
```

Before going further, make sure **index.html** is loading your new classes. Add the two lines below the first `<script>` tag:

```
<script src="//cdn.jsdelivr.net/npm/phaser@3.21.0/dist/phaser.js"></script>
<script src="assets/js/classes/Player.js" charset="utf-8"></script>
<script src="assets/js/classes/Chest.js" charset="utf-8"></script>
<script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
<script src="assets/js/main.js" charset="utf-8"></script>
```

Next go to **GameScene.js** and move the **update()** method to **Player.js**. We are also going to move the lines that setScale and collide with worldBounds. Once that is done we are going to change references to **this.player** to **this.body** (since we are already in the Player class). For convenience we are going to put the player speed into a **velocity** variable and change references to 160 to **this.velocity** instead. Lastly, we are going to have the **update()** method take a **cursors** parameter and change all references to **this.cursors** to simply **cursors**. The Player class should now look like this:

```
class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);

    this.scene = scene;
    this.velocity = 160;

    this.setScale(2);
    this.body.setColliderWorldBounds(true);
  }

  update(cursors) {
    this.body.setVelocity(0);

    if(cursors.left.isDown) {
      this.body.setVelocityX(-this.velocity);
    } else if (cursors.right.isDown) {
      this.body.setVelocityX(this.velocity);
    }

    if(cursors.up.isDown) {
```



```
        this.body.setVelocityY(-this.velocity);
    } else if (cursors.down.isDown) {
        this.body.setVelocityY(this.velocity);
    }
}
}
```

Back in **GameScene.js** make the **update()** method pass the cursors to the Player **update()**:

```
update() {
    this.player.update(this.cursors);
}
```

In **Player.js** change the constructor to enable physics and add the player object to the scene:

```
constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);

    this.scene = scene;
    this.velocity = 160;

    this.scene.physics.world.enable(this); // NEW
    this.setImmovable(false); // NEW

    this.setScale(2);
    this.setCollideWorldBounds(true);
    this.scene.add.existing(this); // NEW
}
```

Next, in the **create()** method of GameScene replace the lines dealing with the player with this one:

```
this.wall = this.physics.add.image(500, 100, 'button1');
this.wall.setImmovable();

this.player = new Player(this, 32, 32, 'characters', 0); // NEW

this.physics.add.collider(this.player, this.wall);
this.physics.add.overlap(this.player, this.chest, function(player, chest) { goldPicku
pAudio.play(); chest.destroy(); });
```

Go ahead and save and refresh. You guessed it. The game should **still** run exactly the same, but at least we are getting organized! ☐☐

In **Chest.js** enter the following:



```
class Chest extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);

    this.scene = scene;
    this.coins = 10;

    this.scene.physics.world.enable(this);
    this.scene.add.existing(this);
  }
}
```

Back in the GameScene **create()** method change the line that creates the chest:

```
var button = this.add.image(10, 100, 'button1');
button.setOrigin(0,0);

this.chest = new Chest(this, 300, 300, 'items', 0); // NEW

this.wall = this.physics.add.image(500, 100, 'button1');
this.wall.setImmovable();
```

Save and refresh. You can guess what you should still see...



Now to work on the TitleScene.

First modify the **create()** method:

```
create() {
    this.titleText = this.add.text(this.scale.width / 2, this.scale.height / 2, 'Zenva MMORPG', {fontSize: '64px', fill: '#fff' }); this.titleText.setOrigin(0.5); // NEW
    this.button = this.add.image(this.scale.width / 2, this.scale.height * 0.65, 'button1');
}
```

In the BootScene have **create()** call the TitleScene instead of the GameScene:

```
create() {
    this.scene.start('Title');
}
```

If you save and refresh you should just see “Zenva MMORPG” and a button below that.

Now we’ll jazz up the button a little bit. In BootScene **preload()** load in a new asset:

```
preload() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    this.load.image('button2', 'assets/images/ui/blue_button02.png'); // NEW
    // etc ...
}
```

Then in TitleScene make the button interactive, give it some text and listen for pointer events to respond to:

```
create() {
    this.titleText = this.add.text(this.scale.width / 2, this.scale.height / 2, 'Zenva MMORPG', {fontSize: '64px', fill: '#fff' });
    this.titleText.setOrigin(0.5);

    this.button = this.add.image(this.scale.width / 2, this.scale.height * 0.65, 'button1');
    // NEW code starts here
    this.button.setInteractive();

    this.buttonText = this.add.text(0,0, 'Start', {fontSize: '26px', fill:'#fff'});
    Phaser.Display.Align.In.Center(this.buttonText, this.button);

    this.button.on('pointerdown', () => {
        this.scene.start('Game');
    });

    this.button.on('pointerover', () => {
        this.button.setTexture('button2');
    });
}
```



```
this.button.on('pointerout', () => {  
    this.button.setTexture('button1');  
});  
// NEW code ends here  
}
```

When you save and refresh, the button should now have a pointer hover effect and when you click it your GameScene should load.



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

Now we are going to do some refactoring to make it easier to create buttons when we need them.

In your **classes** subfolder create a new file called **UIButton.js** and inside that create a class definition and a constructor that stores all the parameters:

```
class UIButton extends Phaser.GameObjects.Container {
  constructor(scene, x, y, key, hoverKey, text, targetCallback) {
    super(scene, x, y);
    this.scene = scene;
    this.x = x;
    this.y = y;
    this.key = key;
    this.hoverKey = hoverKey;
    this.text = text;
    this.targetCallback = targetCallback;
  }
}
```

Inside the constructor at the bottom, add the following (note that we haven't coded `createButton()` yet):

```
this.createButton();
this.scene.add.existing(this);
```

Before we forget, let's go to **index.html** and load in this new class (it's the third line down):

```
<script src="assets/js/classes/Player.js" charset="utf-8"></script>
<script src="assets/js/classes/Chest.js" charset="utf-8"></script>
<script src="assets/js/classes/UIButton.js" charset="utf-8"></script>
<script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
<script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
<script src="assets/js/main.js" charset="utf-8"></script>
```

Go to `TitleScreen` and cut all the lines having to do with the button and paste them into a **`createButton()`** method in `UIButton`:

```
createButton() {
  this.button = this.add.image(this.scale.width / 2, this.scale.height * 0.65, 'button1');
  this.button.setInteractive();

  this.buttonText = this.add.text(0,0, 'Start', {fontSize: '26px', fill:'#fff'});
  Phaser.Display.Align.In.Center(this.buttonText, this.button);
}
```



```
this.button.on('pointerdown', () => {
    this.scene.start('Game');
});

this.button.on('pointerover', () => {
    this.button.setTexture('button2');
});

this.button.on('pointerout', () => {
    this.button.setTexture('button1');
});
}
```

Now make the following changes to **createButton()**. There are many new lines so check your code carefully. **The following code has been updated, and differs from the video. Notice that “this.add.image” and “this.add.text” should be changed to “this.scene.add.image” and “this.scene.add.text”:**

```
createButton() {
    this.button = this.scene.add.image(0, 0, this.key);
    this.button.setInteractive();
    this.button.setScale(1.4);

    this.buttonText = this.scene.add.text(0, 0, this.text, { fontSize: '26px', fill:
    '#fff' });
    Phaser.Display.Align.In.Center(this.buttonText, this.button);

    this.add(this.button);
    this.add(this.buttonText);

    this.button.on('pointerdown', () => {
        this.targetCallback;
    });

    this.button.on('pointerover', () => {
        this.button.setTexture(this.hoverKey);
    });

    this.button.on('pointerout', () => {
        this.button.setTexture(this.key);
    });
}
```



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

Now to start using our new button class. Your completed UIButton class should now look like this.

The following code has been updated, and differs from the video:

```
class UIButton extends Phaser.GameObjects.Container {
    constructor(scene, x, y, key, hoverKey, text, targetCallback) {
        super(scene, x, y);
        this.scene = scene; // the scene this container will be added to
        this.x = x; // the x position of our container
        this.y = y; // the y position of our container
        this.key = key; // the background image of our button
        this.hoverKey = hoverKey; // the image that will be displayed when the player h
overs over the button
        this.text = text; // the text that will be displayed on the button
        this.targetCallback = targetCallback; // the callback function that will be cal
led when the player clicks the button

        this.createButton();
        this.scene.add.existing(this);
    }

    createButton() {
        this.button = this.scene.add.image(0, 0, 'button1');
        this.button.setInteractive();
        this.button.setScale(1.4);

        this.buttonText = this.scene.add.text(0, 0, this.text, { fontSize: '26px', fill
: '#fff' });
        Phaser.Display.Align.In.Center(this.buttonText, this.button);

        this.add(this.button);
        this.add(this.buttonText);

        this.button.on('pointerdown', () => {
            this.targetCallback();
        });

        this.button.on('pointerover', () => {
            this.button.setTexture(this.hoverKey);
        });

        this.button.on('pointerout', () => {
            this.button.setTexture(this.key);
        });
    }
}
```

Go into your TitleScene and change the line that creates the button in create(). Also, create a callback method for the button called startScene():



```
class TitleScene extends Phaser.Scene {
    constructor() {
        super('Title');
    }

    create() {
        // create title text
        this.titleText = this.add.text(this.scale.width / 2, this.scale.height / 2, 'Zenva MMORPG', { fontSize: '64px', fill: '#fff' });
        this.titleText.setOrigin(0.5);

        // create the Play game button
        this.startGameButton = new UIButton(this, this.scale.width / 2, this.scale.height * 0.65, 'button1', 'button2', 'Start', this.startScene.bind(this, 'Game')); // NEW
    }

    // NEW function:
    startScene(targetScene) {
        this.scene.start(targetScene);
    }
}
```

When you save and refresh you should still get a button with a hover state that loads the Game scene when you click it.



Phaser can run multiple scenes at one time. We are going to work on the UiScene to get it to overlay the number of coins the player has.

First, go into your UiScene class and delete the **preload()** method. Create stub methods for **setupUiElements()** and **setupEvents()**. Make calls to these stubs inside **create()**:

```
class UiScene extends Phaser.Scene {  
  
    constructor() {  
        super('Ui');  
    }  
  
    create() {  
        this.setupUiElements();  
        this.setupEvents();  
    }  
  
    setupUiElements() {  
  
    }  
  
    setupEvents() {  
  
    }  
}
```

Go ahead and load some content in **setupUiElements()**:

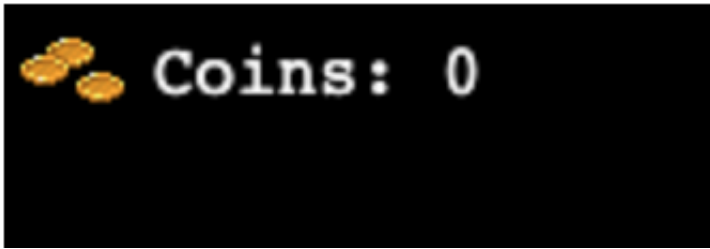
```
setupUiElements() {  
    this.scoreText = this.add.text(35, 8, 'Coins: 0', {fontSize: '16px', fill: '#fff'}  
}); // NEW  
    this.coinIcon = this.add.image(15, 15, 'items', 3); // NEW  
}
```

In your GameScene add an **init()** method that will start the UiScene:

```
init() {  
    this.scene.launch('Ui');  
}
```

NOTE: when you use **launch()** instead of **start()** Phaser will run the scene in parallel.

Now if you save and refresh when you get to your GameScene you should see an overlay with an icon and text in the upper left:





In your `GameScene` **create()** change these lines to allow us to do some refactoring:

```
create() {
    this.goldPickupAudio = this.sound.add('goldSound'); // NEW

    this.chest = new Chest(this, 300, 300, 'items', 0);

    this.wall = this.physics.add.image(500, 100, 'button1');
    this.wall.setImmovable();

    this.player = new Player(this, 32, 32, 'characters', 0);

    this.physics.add.collider(this.player, this.wall);
    this.physics.add.overlap(this.player, this.chest, this.collectChest, null, this);
    // NEW

    this.cursors = this.input.keyboard.createCursorKeys();
}
```

Also add a **collectChest()** method that plays audio and emits an `updateScore` event that carries with it the amount your score goes up:

```
collectChest(player, chest) {
    this.goldPickupAudio.play();
    this.events.emit('updateScore', chest.coins);
    chest.destroy();
}
```

Back in `UiScene` add an **init()** method that gets a reference to the `GameScene`:

```
init() {
    this.gameScene = this.scene.get('Game');
}
```

Then in **setupEvents()** start listening for the `updateScore` event. Create an anonymous function to update the text on screen:

```
setupEvents() {
    this.gameScene.events.on('updateScore', (score) => {
        this.scoreText.setText(`Coins: ${score}`);
    });
}
```

Save and refresh and watch your score go up when you get the chest!



It's a very good thing to stay organized from the beginning so you don't get lost as your project grows. Let's refactor a bit. In the BootScene create 3 loading methods that organize by asset type. Make calls to these in **preload()**:

```
class BootScene extends Phaser.Scene {

    constructor() {
        super('Boot');
    }

    preload() {
        this.loadImages();
        this.loadSpritesheets();
        this.loadAudio();
    }

    loadImages() {
        this.load.image('button1', 'assets/images/ui/blue_button01.png');
        this.load.image('button2', 'assets/images/ui/blue_button02.png');
    }

    loadSpritesheets() {
        this.load.spritesheet('items', 'assets/images/items.png', {frameWidth:32, frameHeight:32});
        this.load.spritesheet('characters', 'assets/images/characters.png', {frameWidth:32, frameHeight:32});
    }

    loadAudio() {
        this.load.audio('goldSound', ['assets/audio/pickup.wav']);
    }

    create() {
        this.scene.start('Title');
    }
}
```

Switch over to GameScene and create methods to organize the content. Be sure to make calls to these new methods in **create()**:

```
class GameScene extends Phaser.Scene {

    constructor() {
        super('Game');
    }

    init() {
        this.scene.launch('Ui');
    }

    create() {
        this.createAudio();
    }
}
```



```
        this.createChests();

        this.createWalls();

        this.createPlayer();

        this.addCollisions();

        this.createInput();
    }

    update() {
        this.player.update(this.cursors);
    }

    createAudio() {
        this.goldPickupAudio = this.sound.add('goldSound');
    }

    createPlayer(){
        this.player = new Player(this, 32, 32, 'characters', 0);
    }

    createChests(){
        this.chest = new Chest(this, 300, 300, 'items', 0);
    }

    createWalls(){
        this.wall = this.physics.add.image(500, 100, 'button1');
        this.wall.setImmovable();
    }

    createInput() {
        this.cursors = this.input.keyboard.createCursorKeys();
    }

    addCollisions() {
        this.physics.add.collider(this.player, this.wall);
        this.physics.add.overlap(this.player, this.chest, this.collectChest, null, this);
    }

    collectChest(player, chest) {
        this.goldPickupAudio.play();
        this.events.emit('updateScore', chest.coins);
        chest.destroy();
    }
}
```

Once that's done, save and refresh to make sure your game still works.



In GameScene make the following changes to spawn multiple chests and replace them when they are picked up:

```
class GameScene extends Phaser.Scene {

    constructor() {
        super('Game');
    }

    init() {
        this.scene.launch('Ui');
        this.score = 0;
    }

    create() {
        this.createAudio();

        this.createChests();

        this.createWalls();

        this.createPlayer();

        this.addCollisions();

        this.createInput();
    }

    update() {
        this.player.update(this.cursors);
    }

    createAudio() {
        this.goldPickupAudio = this.sound.add('goldSound');
    }

    createPlayer(){
        this.player = new Player(this, 32, 32, 'characters', 0);
    }

    createChests(){
        this.chests = this.physics.add.group(); // NEW
        this.chestPositions = [[100,100], [200,200], [300,300], [400,400],[500,500]];
// NEW
        this.maxNumberOfChests = 3; // NEW

        for(let i = 0; i < this.maxNumberOfChests; i++) { // NEW
            this.spawnChest(); // NEW
        } // NEW
    }

    spawnChest() {
        // NEW code starts here
        const location = this.chestPositions[Math.floor(Math.random() * this.chestPositions.length)];
```



```
        this.chest = new Chest(this, location[0], location[1], 'items', 0);
        this.chests.add(this.chest);
// NEW code ends here
    }

    createWalls(){
        this.wall = this.physics.add.image(500, 100, 'button1');
        this.wall.setImmovable();
    }

    createInput() {
        this.cursors = this.input.keyboard.createCursorKeys();
    }

    addCollisions() {
        this.physics.add.collider(this.player, this.wall);
        this.physics.add.overlap(this.player, this.chests, this.collectChest, null, t
his);
    }

    collectChest(player, chest) {
        // NEW code starts here
        this.goldPickupAudio.play();
        this.score += chest.coins;
        this.events.emit('updateScore', this.score);
        chest.destroy();
        this.time.delayedCall(1000, this.spawnChest, [], this);
        // NEW code ends here
    }
}
```

Save and refresh and you should be able to keep spawning multiple chests and have your score go up as you collect them.



Object pooling lets our game be much more efficient by reducing the number of objects spawned at runtime.

To switch over to object pooling, begin by going into your GameScene and changing **spawnChest()**:

```
spawnChest() {
    const location = this.chestPositions[Math.floor(Math.random() * this.chestPositions.length)];

    let chest = this.chests.getFirstDead();

    if(!chest) {
        // if no inactive game object, make one
        const chest = new Chest(this, location[0], location[1], 'items', 0);

        this.chests.add(this.chest);
    } else {
        chest.setPosition(location[0], location[1]);
        chest.makeActive();
    }
}
```

Further down, when you collect a chest, instead of destroying it, we will now set it to inactive:

```
collectChest(player, chest) {
    this.goldPickupAudio.play();
    this.score += chest.coins;
    this.events.emit('updateScore', this.score);
    chest.makeInactive(); // NEW
    this.time.delayedCall(1000, this.spawnChest, [], this);
}
```

Now to make all this work we have to go into the **Chest** class and add two methods:

```
makeActive() {
    this.setActive(true);
    this.setVisible(true);
    this.body.checkCollision.none = false;
}

makeInactive() {
    this.setActive(false);
    this.setVisible(false);
    this.body.checkCollision.none = true;
}
```

Your complete GameScene should now look like:



```
class GameScene extends Phaser.Scene {
  constructor() {
    super('Game');
  }

  init() {
    this.scene.launch('Ui');
    this.score = 0;
  }

  create() {
    this.createAudio();
    this.createChests();
    this.createWalls();
    this.createPlayer();
    this.addCollisions();
    this.createInput();
  }

  update() {
    this.player.update(this.cursors);
  }

  createAudio() {
    this.goldPickupAudio = this.sound.add('goldSound', { loop: false, volume: 0.2 }
);
  }

  createPlayer() {
    this.player = new Player(this, 32, 32, 'characters', 0);
  }

  createChests() {
    // create a chest group
    this.chests = this.physics.add.group();
    // create chest positions array
    this.chestPositions = [[100, 100], [200, 200], [300, 300], [400, 400], [500, 50
0]];
    // specify the max number of chest we can have
    this.maxNumberOfChests = 3;
    // spawn a chest
    for (let i = 0; i < this.maxNumberOfChests; i += 1) {
      this.spawnChest();
    }
  }

  spawnChest() {
    const location = this.chestPositions[Math.floor(Math.random() * this.chestPosit
ions.length)];

    let chest = this.chests.getFirstDead();

    if (!chest) {
      const chest = new Chest(this, location[0], location[1], 'items', 0);
      // add chest to chests group
    }
  }
}
```



```
        this.chests.add(chest);
    } else {
        chest.setPosition(location[0], location[1]);
        chest.makeActive();
    }
}

createWalls() {
    this.wall = this.physics.add.image(500, 100, 'button1');
    this.wall.setImmovable();
}

createInput() {
    this.cursors = this.input.keyboard.createCursorKeys();
}

addCollisions() {
    // check for collisions between player and wall objects
    this.physics.add.collider(this.player, this.wall);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, thi
s);
}

collectChest(player, chest) {
    // play gold pickup sound
    this.goldPickupAudio.play();
    // update our score
    this.score += chest.coins;
    // update score in the ui
    this.events.emit('updateScore', this.score);
    // make chest game object inactive
    chest.makeInactive();
    // spawn a new chest
    this.time.delayedCall(1000, this.spawnChest, [], this);
}
}
```

And your complete Chest class should look like:

```
class Chest extends Phaser.Physics.Arcade.Image {
    constructor(scene, x, y, key, frame) {
        super(scene, x, y, key, frame);
        this.scene = scene; // the scene this game object will be added to
        this.coins = 10; // the amount of coins this chest contains

        // enable physics
        this.scene.physics.world.enable(this);
        // add the player to our existing scene
        this.scene.add.existing(this);
    }
}
```



```
makeActive() {  
    this.setActive(true);  
    this.setVisible(true);  
    this.body.checkCollision.none = false;  
}  
  
makeInactive() {  
    this.setActive(false);  
    this.setVisible(false);  
    this.body.checkCollision.none = true;  
}  
}
```

Save and refresh your newly efficient game!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Index.html

Found in the Project root folder

This HTML code sets up a simple web page for a game using the Phaser library. It initializes the viewport, and imports multiple JavaScript files that contain various game classes and scenes, including player and UI elements.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="user-scalable=0, initial-scale=1, minimum-
scale=1, maximum-scale=1, width=device-width, minimal-ui=1">
    <title>Zenva MMORPG</title>
  </head>
  <body>
    <div id="phaser-game"></div>
    <script src="assets/js/lib/phaser.min.js" charset="utf-8"></script>
    <script src="assets/js/classes/Player.js" charset="utf-8"></script>
    <script src="assets/js/classes/Chest.js" charset="utf-8"></script>
    <script src="assets/js/classes/UiButton.js" charset="utf-8"></script>
    <script src="assets/js/scenes/BootScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/TitleScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/GameScene.js" charset="utf-8"></script>
    <script src="assets/js/scenes/UiScene.js" charset="utf-8"></script>
    <script src="assets/js/main.js" charset="utf-8"></script>
  </body>
</html>
```

Main.js

Found in Project/assets/js

This code sets up the configuration for a Phaser game with dimensions 800×600, including arcade physics with no gravitational effect. It specifies four game scenes: BootScene, TitleScene, GameScene, and UiScene. It also enables debugging. Finally, it initiates the game with the created configuration.

```
var config = {
  type: Phaser.AUTO,
  width: 800,
  height: 600,
  scene: [
    BootScene,
    TitleScene,
```



```
    GameScene,
    UiScene,
  ],
  physics: {
    default: 'arcade',
    arcade: {
      debug: true,
      gravity: {
        y: 0,
      },
    },
  },
},
};

var game = new Phaser.Game(config);
```

Chest.js

Found in Project/assets/js/classes

This code defines a 'Chest' class in a video game implemented using Phaser. The class extends 'Phaser.Physics.Arcade.Image' to enable physics simulations. This chest object holds coins and has methods to activate or deactivate its visibility and collision in the game scene.

```
class Chest extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);
    this.scene = scene; // the scene this game object will be added to
    this.coins = 10; // the amount of coins this chest contains

    // enable physics
    this.scene.physics.world.enable(this);
    // add the player to our existing scene
    this.scene.add.existing(this);
  }

  makeActive() {
    this.setActive(true);
    this.setVisible(true);
    this.body.checkCollision.none = false;
  }

  makeInactive() {
    this.setActive(false);
    this.setVisible(false);
    this.body.checkCollision.none = true;
  }
}
```

Player.js



Found in Project/assets/js/classes

This code defines a Player class for a game, which extends Phaser.Physics.Arcade.Image, indicating that it is a physics-enabled graphical game object. The constructor initializes properties such as the scene it belongs to, its speed, and its interaction with other objects and the world bounds. The update function defines the player's movements based on input from the cursors (arrow keys).

```
class Player extends Phaser.Physics.Arcade.Image {
  constructor(scene, x, y, key, frame) {
    super(scene, x, y, key, frame);
    this.scene = scene; // the scene this container will be added to
    this.velocity = 160; // the velocity when moving our player

    // enable physics
    this.scene.physics.world.enable(this);
    // set immovable if another object collides with our player
    this.setImmovable(false);
    // scale our player
    this.setScale(2);
    // collide with world bounds
    this.setCollideWorldBounds(true);
    // add the player to our existing scene
    this.scene.add.existing(this);
  }

  update(cursors) {
    this.body.setVelocity(0);

    if (cursors.left.isDown) {
      this.body.setVelocityX(-this.velocity);
    } else if (cursors.right.isDown) {
      this.body.setVelocityX(this.velocity);
    }

    if (cursors.up.isDown) {
      this.body.setVelocityY(-this.velocity);
    } else if (cursors.down.isDown) {
      this.body.setVelocityY(this.velocity);
    }
  }
}
```

UIButton.js

Found in Project/assets/js/classes

The code defines a UIButton class that extends the functionalities of the Phaser's GameObjects.Container. This class helps to create interactive buttons, with a specified position, key, hover key, and a callback function. It also handles button interactivity events, such as clicking, hovering over, and moving the cursor away from the button.

```
class UIButton extends Phaser.GameObjects.Container {
  constructor(scene, x, y, key, hoverKey, text, targetCallback) {
```



```
super(scene, x, y);
this.scene = scene; // the scene this container will be added to
this.x = x; // the x position of our container
this.y = y; // the y position of our container
this.key = key; // the background image of our button
this.hoverKey = hoverKey; // the image that will be displayed when the player hovers over the button
this.text = text; // the text that will be displayed on the button
this.targetCallback = targetCallback; // the callback function that will be called when the player clicks the button

// create our Ui Button
this.createButton();
// add this container to our Phaser Scene
this.scene.add.existing(this);
}

createButton() {
  // create play game button
  this.button = this.scene.add.image(0, 0, 'button1');
  // make button interactive
  this.button.setInteractive();
  // scale the button
  this.button.setScale(1.4);

  // create the button text
  this.buttonText = this.scene.add.text(0, 0, this.text, { fontSize: '26px', fill: '#fff' });
  // center the button text inside the Ui button
  Phaser.Display.Align.In.Center(this.buttonText, this.button);

  // add the two game objects to our container
  this.add(this.button);
  this.add(this.buttonText);

  // listen for events
  this.button.on('pointerdown', () => {
    this.targetCallback();
  });

  this.button.on('pointerover', () => {
    this.button.setTexture(this.hoverKey);
  });

  this.button.on('pointerout', () => {
    this.button.setTexture(this.key);
  });
}
```

BootScene.js

Found in Project/assets/js/scenes

This code defines the BootScene class in the Phaser game framework. It has methods for preloading assets including images, sprite sheets, and audio. After loading these assets, the 'Title' scene is then started.

```
class BootScene extends Phaser.Scene {
  constructor() {
    super('Boot');
  }

  preload() {
    // load images
    this.loadImages();
    // load spritesheets
    this.loadSpriteSheets();
    // load audio
    this.loadAudio();
  }

  loadImages() {
    this.load.image('button1', 'assets/images/ui/blue_button01.png');
    this.load.image('button2', 'assets/images/ui/blue_button02.png');
  }

  loadSpriteSheets() {
    this.load.spritesheet('items', 'assets/images/items.png', { frameWidth: 32, frameHeight: 32 });
    this.load.spritesheet('characters', 'assets/images/characters.png', { frameWidth: 32, frameHeight: 32 });
  }

  loadAudio() {
    this.load.audio('goldSound', ['assets/audio/Pickup.wav']);
  }

  create() {
    this.scene.start('Title');
  }
}
```

GameScene.js

Found in Project/assets/js/scenes

This `GameScene` class, extending `Phaser.Scene`, defines all the key aspects of the gameplay within a Phaser gaming framework – the initialization (including launching UI scene and setting initial score), creating key objects like player, chests, walls, handling collisions, and updating player movements. It also details unique events that happen when a player collects a chest, including awarding points, and spawning a new chest.

```
class GameScene extends Phaser.Scene {
  constructor() {
    super('Game');
  }
}
```



```
init() {
  this.scene.launch('Ui');
  this.score = 0;
}

create() {
  this.createAudio();
  this.createChests();
  this.createWalls();
  this.createPlayer();
  this.addCollisions();
  this.createInput();
}

update() {
  this.player.update(this.cursors);
}

createAudio() {
  this.goldPickupAudio = this.sound.add('goldSound', { loop: false, volume: 0.2 });
}

createPlayer() {
  this.player = new Player(this, 32, 32, 'characters', 0);
}

createChests() {
  // create a chest group
  this.chests = this.physics.add.group();
  // create chest positions array
  this.chestPositions = [[100, 100], [200, 200], [300, 300], [400, 400], [500, 500]
];
  // specify the max number of chest we can have
  this.maxNumberOfChests = 3;
  // spawn a chest
  for (let i = 0; i < this.maxNumberOfChests; i += 1) {
    this.spawnChest();
  }
}

spawnChest() {
  const location = this.chestPositions[Math.floor(Math.random() * this.chestPositions.length)];

  let chest = this.chests.getFirstDead();

  if (!chest) {
    const chest = new Chest(this, location[0], location[1], 'items', 0);
    // add chest to chests group
    this.chests.add(chest);
  } else {
    chest.setPosition(location[0], location[1]);
    chest.makeActive();
  }
}
```



```
}

createWalls() {
    this.wall = this.physics.add.image(500, 100, 'button1');
    this.wall.setImmovable();
}

createInput() {
    this.cursors = this.input.keyboard.createCursorKeys();
}

addCollisions() {
    // check for collisions between player and wall objects
    this.physics.add.collider(this.player, this.wall);
    // check for overlaps between player and chest game objects
    this.physics.add.overlap(this.player, this.chests, this.collectChest, null, this)
;
}

collectChest(player, chest) {
    // play gold pickup sound
    this.goldPickupAudio.play();
    // update our score
    this.score += chest.coins;
    // update score in the ui
    this.events.emit('updateScore', this.score);
    // make chest game object inactive
    chest.makeInactive();
    // spawn a new chest
    this.time.delayedCall(1000, this.spawnChest, [], this);
}
}
```

TitleScene.js

Found in Project/assets/js/scenes

This code defines a title scene for a game using Phaser. It sets up the title text and a “Start” button; clicking the button leads to the start of the game.

```
class TitleScene extends Phaser.Scene {
    constructor() {
        super('Title');
    }

    create() {
        // create title text
        this.titleText = this.add.text(this.scale.width / 2, this.scale.height / 2, 'Zenv
a MMORPG', { fontSize: '64px', fill: '#fff' });
        this.titleText.setOrigin(0.5);

        // create the Play game button
        this.startGameButton = new UIButton(this, this.scale.width / 2, this.scale.height
```



```
* 0.65, 'button1', 'button2', 'Start', this.startScene.bind(this, 'Game'));
}

startScene(targetScene) {
    this.scene.start(targetScene);
}
}
```

UiScene.js

Found in Project/assets/js/scenes

The given code demonstrates the setup of a User Interface (UI) scene in a game using Phaser. It establishes a link to the game scene, sets up UI elements (a score count and a coin icon), and sets up an event to update the score when coins are collected in the game.

```
class UiScene extends Phaser.Scene {
    constructor() {
        super('Ui');
    }

    init() {
        // grab a reference to the game scene
        this.gameScene = this.scene.get('Game');
    }

    create() {
        this.setupUiElements();
        this.setupEvents();
    }

    setupUiElements() {
        // create the score text game object
        this.scoreText = this.add.text(35, 8, 'Coins: 0', { fontSize: '16px', fill: '#fff'
    });
        // create coin icon
        this.coinIcon = this.add.image(15, 15, 'items', 3);
    }

    setupEvents() {
        // listen for the updateScore event from the game scene
        this.gameScene.events.on('updateScore', (score) => {
            this.scoreText.setText(`Coins: ${score}`);
        });
    }
}
```